📄 Document de référence technique — Projet intégrateur PIL1

IFRI_MentorLink

Guide technique complet pour la conception et le développement de la plateforme de mentorat académique de l'IFRI. Ce document couvre l'ensemble du projet : architecture, base de données, répartition des tâches et workflow de développement.

ÉTABLISSEMENT	NIVEAU	DEADLINE RENDU	STACK TECHNIQUE
IFRI — UAC, Cotonou	Licence 1 (PIL1 2025-2026)	Mercredi 10 juin 2026, 23h59	Vue.js · FastAPI · Supabase/PostgreSQL

1 Vue d'ensemble du projet

Contexte et objectif

IFRI_MentorLink est une application web de mise en relation entre étudiants de l'IFRI. Son objectif est simple : permettre à un étudiant qui maîtrise une matière (le **mentor**) d'aider un autre qui en a besoin (le **mentoré**), de façon organisée et tracée via une plateforme numérique.

Ce projet est votre projet intégrateur de première année. Il mobilise toutes les compétences acquises : algorithmique, développement web, bases de données relationnelles, Python et collaboration en équipe avec Git. Ce n'est pas un exercice isolé — c'est un projet complet, du même type que ceux réalisés en entreprise.

Contrainte critique : vous avez **4 jours effectifs** (vendredi → lundi) pour finir le développement. Le rendu final est le mercredi 10 juin à 23h59. Aucun retard ne sera accepté et aucun projet rendu en retard ne sera évalué.

Les 3 modules principaux

MODULE 1
Gestion des comptes et profils
Inscription, connexion, profil utilisateur avec compétences, lacunes et disponibilités.

MODULE 2
Matching mentor ↔ mentoré
Offres/demandes de mentorat et algorithme de mise en correspondance automatique avec score.

MODULE 3
Messagerie instantanée
Chat en temps réel entre mentors et mentorés, avec historique et notifications.

Contraintes techniques imposées par le cahier de charges

CONTRAINTE	DÉTAIL	STATUT
Backend	Python obligatoire (Django, Flask, FastAPI) — PHP, Node.js, Java **interdits**	✓ Respecté (FastAPI)
Frontend	HTML/CSS/JS — Bootstrap, Tailwind, Vue.js autorisés	✓ Respecté (Vue.js)
Base de données	SGBD relationnel SQL — MySQL ou PostgreSQL. SQLite déconseillé	✓ Respecté (PostgreSQL/Supabase)
Git	Tous les membres doivent committer régulièrement sur GitHub	⚠ À respecter impérativement
Livrables	Code source + rapport HTML + fichier .sql	→ Voir section 7
Sécurité	Mots de passe hashés, authentification sécurisée	→ Voir section 5

Le dépôt GitHub doit s'appeler `PIL1_2526_[NUMERO_DU_GROUPE]` et les comptes GitHub `ratheilesse` , `primearwyn` et `MaryseGAHOU` doivent avoir accès au dépôt.

2 Parcours utilisateur

Comprendre le parcours de l'utilisateur est fondamental avant d'écrire la première ligne de code. Il permet de savoir quels écrans créer, quels endpoints développer et comment la base de données doit être structurée. Voici le parcours complet d'un étudiant sur IFRI_MentorLink.

Point important : un utilisateur peut être **à la fois mentor ET mentoré** sur la plateforme. Par exemple, un étudiant fort en Python mais faible en algèbre peut offrir du mentorat en Python et en demander en algèbre simultanément.

1 **Découverte et inscription**

L'étudiant arrive sur la **page d'accueil (landing page)** et clique sur « S'inscrire ». Il remplit le formulaire d'inscription avec :

- Informations d'identité : nom, prénom, adresse email, numéro de téléphone
- Mot de passe (qui sera **haché côté serveur**, jamais stocké en clair)
- Sa filière (IA, IM, GL, SE&IoT, SI) et son niveau d'études
- Ses **points forts** (compétences maîtrisées) : ex. Python, SQL, Algorithmique
- Ses **points faibles** (matières où il a besoin d'aide) : ex. Réseaux, Mathématiques
- Ses **disponibilités horaires** (jours de la semaine + plages horaires)
- Une bio courte et ses centres d'intérêt académiques (optionnel)

Le système vérifie que l'email et le numéro de téléphone ne sont pas déjà utilisés, puis crée le compte.

2 **Connexion et accès au tableau de bord**

L'étudiant se connecte avec son email (ou téléphone) + mot de passe. Le système vérifie les identifiants, génère un **token JWT** et renvoie l'utilisateur vers son **tableau de bord personnel (dashboard)**.

En cas d'oubli du mot de passe, une option de réinitialisation est disponible. Le dashboard est la page centrale de l'application : c'est de là que tout se fait.

3 **Publication d'offres et demandes de mentorat**

Depuis son dashboard, l'étudiant peut créer des **posts de mentorat**. Il y a deux types :

- **Offre de mentorat** : "Je peux aider en SQL — disponible le mercredi soir — en ligne"
- **Demande de mentorat** : "J'ai besoin d'aide en Algorithmique — disponible le samedi matin — présentiel ou en ligne"

Chaque post précise la matière concernée, les créneaux horaires et le format (présentiel / en ligne / les deux). Un utilisateur peut avoir plusieurs posts actifs en même temps.

Il peut aussi **parcourir les offres/demandes des autres étudiants** et répondre à celles qui correspondent à ses compétences ou besoins.

4 **Consultation des correspondances (matching)**

Le cœur de la plateforme. L'algorithme de matching analyse les profils et les posts pour calculer automatiquement un **score de compatibilité** entre chaque paire mentor-mentoré potentielle.

Les résultats affichent pour chaque correspondance : la photo et le nom du mentor/mentoré, les compétences en commun, les créneaux disponibles partagés, et le score de compatibilité sur 100.

L'étudiant peut **accepter ou refuser** une correspondance proposée.

5 **Messagerie et organisation des sessions**

Lorsqu'un match est **accepté par les deux parties**, une conversation est automatiquement créée entre eux. Ils peuvent alors :

- S'envoyer des messages textuels en temps réel
- Recevoir des notifications instantanées lors d'un nouveau message
- Consulter l'historique complet de leur conversation

⚠ La messagerie n'est disponible **qu'entre utilisateurs ayant un match accepté**. On ne peut pas envoyer de message à n'importe qui.

6 **Utilisation continue**

Au fil du temps, l'étudiant peut modifier son profil (nouvelles compétences acquises, disponibilités qui changent), désactiver/supprimer des posts, consulter de nouvelles correspondances et gérer ses conversations en cours.

3 Choix technologiques

Les technologies choisies sont **Vue.js** (frontend), **FastAPI** (backend Python) et **Supabase/PostgreSQL** (base de données hébergée). Ce choix n'est pas arbitraire — voici pourquoi chaque outil a été retenu et comment ils fonctionnent ensemble.

Vue.js 3

Framework JavaScript progressif pour construire des interfaces utilisateur réactives.

POURQUOI VUE.JS ?

- ✓ Syntaxe simple et lisible, idéale pour des débutants
- ✓ Composants réutilisables (Navbar, ChatBox, etc.)
- ✓ Réactivité automatique : l'UI se met à jour dès que les données changent
- ✓ Autorisé explicitement par le cahier de charges
- ✓ Intégration facile avec une API REST (axios)

FastAPI

Framework Python moderne et rapide pour construire des APIs REST.

POURQUOI FASTAPI ?

- ✓ Python = obligatoire selon le cahier de charges
- ✓ Documentation API automatique (Swagger UI) → très pratique pour le débogage
- ✓ Validation automatique des données avec Pydantic
- ✓ Gestion du JWT intégrée et simple
- ✓ Code clair, bien structuré, facile à comprendre

Supabase + PostgreSQL

Base de données PostgreSQL hébergée avec repas relatif et interface d'administration web.

POURQUOI SUPABASE ?

- ✓ PostgreSQL en dessous → conforme au cahier de charges
- ✓ Hébergé dans le cloud → pas besoin d'installer MySQL localement
- ✓ Interface web pour gérer la BDD visuellement
- ✓ **Supabase Realtime** → notifications et chat en temps réel sans serveur WebSocket
- ✓ Export SQL disponible pour le livrable .sql

Comment ces trois technologies interagissent

```
🌐 Navigateur (Vue.js) → envoie des requêtes HTTP (JSON) → FastAPI
   Exemple : POST /api/auth/login avec { email, password }
⚡ FastAPI → interroge la base de données → Supabase/PostgreSQL
   Exemple : SELECT * FROM users WHERE email = '...' AND password_hash = '...'
↩ FastAPI → renvoie une réponse JSON → Vue.js met à jour l'interface
   Exemple : { token: "eyJhb...", user: { id, name, ... } }
⚡ Pour le chat en temps réel : Vue.js écoute directement Supabase Realtime
   Supabase notifie automatiquement Vue.js quand un nouveau message est inséré
```

Technologies interdites par le cahier de charges : PHP (Laravel, Symfony), Node.js, Ruby on Rails, Bun, Java côté serveur. L'utilisation de SQLite est déconseillée. Ne pas prendre de risque : rester sur la stack Vue.js + FastAPI + PostgreSQL/Supabase.

Outils de développement et collaboration

CÔTÉ BACKEND

- **Python 3.11+** — langage principal
- **pip** — gestionnaire de paquets Python
- **uvicorn** — serveur pour lancer FastAPI
- **psycopg2 / asyncpg** — connexion PostgreSQL
- **python-jose** — gestion des tokens JWT
- **passlib[bcrypt]** — hachage des mots de passe
- **SQLAlchemy** — ORM pour interagir avec la BDD

CÔTÉ FRONTEND

- **Node.js + npm** — pour installer Vue.js
- **Vite** — bundler ultra-rapide pour Vue 3
- **Vue Router** — navigation entre les pages
- **Pinia** — gestion de l'état global (utilisateur connecté, etc.)
- **Axios** — envoi de requêtes HTTP vers l'API FastAPI
- **@supabase/supabase-js** — client Supabase pour le temps réel

4 Modélisation de la base de données

La base de données est le fondement de toute l'application. **Chaque fonctionnalité repose sur au moins une table.** Le schéma qui suit a été conçu spécifiquement pour les besoins de IFRI_MentorLink, en tenant compte des contraintes de débutants et des exigences du cahier de charges. Il y a **9 tables** au total.

Toutes les tables sont en **PostgreSQL**. Le script SQL complet de création est disponible en fin de section. Ce même script constituera votre livrable `.sql` final.

Les 9 tables et leur rôle

TABLE 1 — `USERS` (UTILISATEURS)

C'est la table centrale. Chaque étudiant inscrit sur la plateforme correspond à une ligne dans cette table.

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique auto-incrémenté par PostgreSQL
`first_name`	`VARCHAR(50)`	Prénom de l'étudiant — obligatoire
`last_name`	`VARCHAR(50)`	Nom de famille — obligatoire
`email`	`VARCHAR(100)`	Adresse email — UNIQUE, sert d'identifiant de connexion
`phone_number`	`VARCHAR(20)`	Numéro de téléphone — UNIQUE, sert aussi d'identifiant
`password_hash`	`VARCHAR(255)`	Hash bcrypt du mot de passe — **jamais le mot de passe en clair**
`profile_photo`	`VARCHAR(255)`	URL de la photo de profil — NULL si aucune photo
`field_of_study`	`VARCHAR(50)`	Filière : IA, IM, GL, SE&IoT ou SI
`level`	`VARCHAR(20)`	Niveau d'études : L1, L2, L3, M1, M2
`bio`	`TEXT`	Bio courte et centres d'intérêt académiques — optionnel
`created_at`	`TIMESTAMP`	Date et heure d'inscription — remplie automatiquement
`updated_at`	`TIMESTAMP`	Dernière modification du profil — mise à jour automatiquement

TABLE 2 — `SKILLS` (COMPÉTENCES / MATIÈRES)

Table de référence listant toutes les matières et compétences disponibles sur la plateforme. Elle est pré-remplie avec les matières IFRI et ne change pas souvent.

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique
`name`	`VARCHAR(100)`	Nom de la compétence — UNIQUE. Ex: "Algorithmique", "Python", "SQL"

Exemples de compétences à pré-remplir dans cette table : Algorithmique, Structures de données, Python, Développement Web, JavaScript, SQL, Algèbre relationnelle, Réseaux, Linux, Mathématiques discrètes, Intelligence Artificielle, Vue.js, FastAPI.

TABLE 3 — `USER_SKILLS` (COMPÉTENCES D'UN UTILISATEUR)

Table de liaison entre `users` et `skills` . Elle permet à un étudiant de déclarer ses points forts (compétences maîtrisées) et ses points faibles (lacunes). C'est une relation **many-to-many** avec un attribut supplémentaire.

CHAMP	TYPE	DESCRIPTION
PK/FK `user_id`	`INT`	Référence vers `users.id` — qui est cet utilisateur
PK/FK `skill_id`	`INT`	Référence vers `skills.id` — quelle compétence
`proficiency`	`VARCHAR(10)`	Niveau : `'strong'` = maîtrisé (point fort) ou `'weak'` = lacune (besoin d'aide)

La clé primaire est composée de `(user_id, skill_id)` — un utilisateur ne peut avoir qu'un seul proficiency par compétence.

TABLE 4 — `USER_AVAILABILITIES` (DISPONIBILITÉS DU PROFIL)

Stocke les créneaux de disponibilité habituelle de chaque utilisateur. Utilisée par l'algorithme de matching pour trouver des horaires communs. Un utilisateur peut avoir plusieurs créneaux.

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique du créneau
FK `user_id`	`INT`	Référence vers `users.id`
`day_of_week`	`VARCHAR(10)`	Jour : Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday
`start_time`	`TIME`	Heure de début du créneau. Ex: 14:00
`end_time`	`TIME`	Heure de fin du créneau. Ex: 17:00

Pourquoi une table séparée et pas un champ JSONB ? Une table relationnelle permet d'écrire des requêtes SQL simples pour trouver les créneaux communs entre deux utilisateurs. Avec du JSONB, ce serait beaucoup plus complexe à écrire, surtout en première année.

TABLE 5 — `MENTORSHIP_POSTS` (OFFRES ET DEMANDES)

Un post représente soit une offre ("je peux aider") soit une demande ("j'ai besoin d'aide"). Un utilisateur peut avoir plusieurs posts actifs simultanément.

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique
FK `user_id`	`INT`	Référence vers `users.id` — qui a créé ce post
`type`	`VARCHAR(10)`	`'offer'` = offre de mentorat / `'request'` = demande de mentorat
FK `skill_id`	`INT`	Référence vers `skills.id` — la matière concernée par ce post
`mode`	`VARCHAR(10)`	`'online'` , `'offline'` ou `'both'`
`description`	`TEXT`	Description optionnelle du post
`is_active`	`BOOLEAN`	`TRUE` par défaut. Passer à `FALSE` pour archiver le post sans le supprimer
`created_at`	`TIMESTAMP`	Date de création
`updated_at`	`TIMESTAMP`	Dernière modification

Décision de conception : un post = une seule matière (`skill_id` singulier). Si un utilisateur veut proposer deux matières, il crée deux posts. C'est plus simple à implémenter et plus clair pour l'algorithme de matching.

TABLE 6 — `POST_AVAILABILITIES` (DISPONIBILITÉS D'UN POST)

Les disponibilités spécifiques à un post de mentorat (qui peuvent différer des disponibilités générales du profil). Même structure que `user_availabilities` mais liée à un post.

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique
FK `post_id`	`INT`	Référence vers `mentorship_posts.id`
`day_of_week`	`VARCHAR(10)`	Jour de la semaine
`start_time`	`TIME`	Heure de début
`end_time`	`TIME`	Heure de fin

TABLE 7 — `MATCHES` (CORRESPONDANCES CALCULÉES)

Stocke les paires mentor-mentoré calculées par l'algorithme, avec leur score de compatibilité et leur statut (en attente, accepté, refusé).

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique
FK `mentor_id`	`INT`	Référence vers `users.id` — l'utilisateur qui offre le mentorat
FK `mentee_id`	`INT`	Référence vers `users.id` — l'utilisateur qui reçoit le mentorat
FK `offer_post_id`	`INT`	Post de type 'offer' qui a généré ce match (NULL si match calculé globalement)
FK `request_post_id`	`INT`	Post de type 'request' qui a généré ce match (NULL si calcul global)
FK `skill_id`	`INT`	La compétence commune à ce match
`score`	`DECIMAL(5,2)`	Score de compatibilité calculé, sur 100
`status`	`VARCHAR(20)`	`'pending'` (proposé), `'accepted'` (accepté), `'rejected'` (refusé)
`created_at`	`TIMESTAMP`	Date de création du match

Le champ `status` est crucial. Un match `'pending'` est juste une proposition. La messagerie n'est disponible que pour les matches avec `status = 'accepted'` .

TABLE 8 — `CONVERSATIONS`

Une conversation est créée automatiquement lorsqu'un match est accepté. Il y a exactement **une conversation par match accepté** (relation 1:1).

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique
FK `match_id`	`INT`	Référence vers `matches.id` — UNIQUE (une conv par match)
`created_at`	`TIMESTAMP`	Date de création de la conversation
`updated_at`	`TIMESTAMP`	Date du dernier message (mise à jour à chaque message)

TABLE 9 — `MESSAGES`

Stocke chaque message envoyé dans une conversation. Cette table peut grossir rapidement — c'est normal.

CHAMP	TYPE	DESCRIPTION
PK `id`	`SERIAL`	Identifiant unique
FK `conversation_id`	`INT`	Référence vers `conversations.id`
FK `sender_id`	`INT`	Référence vers `users.id` — qui a envoyé ce message
`content`	`TEXT`	Le contenu textuel du message
`is_read`	`BOOLEAN`	`FALSE` par défaut. Passe à `TRUE` quand le destinataire lit le message
`created_at`	`TIMESTAMP`	Date et heure d'envoi

Relations entre les tables

```
users ──(1:N)──→ user_skills (un user a plusieurs compétences)
skills ──(1:N)──→ user_skills (une compétence est liée à plusieurs users)
users ──(M:N)──→ skills via user_skills (many-to-many avec attribut proficiency)

users ──(1:N)──→ user_availabilities (un user a plusieurs créneaux)

users ──(1:N)──→ mentorship_posts (un user crée plusieurs posts)
skills ──(1:N)──→ mentorship_posts (une compétence sur plusieurs posts)
mentorship_posts ──(1:N)──→ post_availabilities

users(mentor) ──(1:N)──→ matches (un user peut être mentor dans plusieurs matches)
users(mentee) ──(1:N)──→ matches (un user peut être mentoré dans plusieurs matches)

matches ──(1:1)──→ conversations (une conv. par match accepté)
conversations ──(1:N)──→ messages (une conv. contient plusieurs messages)
```

Script SQL complet de création

Ce script doit être exécuté dans Supabase (onglet SQL Editor) pour créer toute la structure. C'est aussi le fichier `.sql` à rendre dans les livrables.

```sql
-- ============================================================
-- IFRI_MentorLink — Script de création de la base de données
-- PostgreSQL / Supabase
-- Projet intégrateur PIL1 2025-2026
-- ============================================================


-- ── TABLE: users ─────────────────────────────────────────
CREATE TABLE users (
    id              SERIAL PRIMARY KEY,
    first_name      VARCHAR(50)  NOT NULL,
    last_name       VARCHAR(50)  NOT NULL,
    email           VARCHAR(100) NOT NULL UNIQUE,
    phone_number    VARCHAR(20)  NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    profile_photo   VARCHAR(255),              -- URL, NULL si pas de photo
    field_of_study  VARCHAR(50)  NOT NULL,     -- IA, IM, GL, SE&IoT, SI
    level           VARCHAR(20)  NOT NULL,     -- L1, L2, L3, M1, M2
    bio             TEXT,
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);


-- ── TABLE: skills ────────────────────────────────────────
CREATE TABLE skills (
    id   SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE   -- ex: "Algorithmique", "Python", "SQL"
);


-- ── TABLE: user_skills ───────────────────────────────────
-- Compétences et lacunes d'un utilisateur (many-to-many avec attribut)
CREATE TABLE user_skills (
    user_id     INT         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    skill_id    INT         NOT NULL REFERENCES skills(id) ON DELETE CASCADE,
    proficiency VARCHAR(10) NOT NULL CHECK (proficiency IN ('strong', 'weak')),
    PRIMARY KEY (user_id, skill_id)
    -- 'strong' = point fort (peut enseigner)
    -- 'weak'   = lacune (a besoin d'aide)
);


-- ── TABLE: user_availabilities ───────────────────────────
-- Disponibilités horaires habituelles de l'utilisateur
CREATE TABLE user_availabilities (
    id          SERIAL PRIMARY KEY,
    user_id     INT         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    day_of_week VARCHAR(10) NOT NULL CHECK (day_of_week IN (
        'Monday','Tuesday','Wednesday','Thursday',
        'Friday','Saturday','Sunday'
    )),
    start_time  TIME NOT NULL,
    end_time    TIME NOT NULL,
    CONSTRAINT chk_user_avail_time CHECK (start_time < end_time)
);


-- ── TABLE: mentorship_posts ──────────────────────────────
-- Offres et demandes de mentorat publiées par les utilisateurs
CREATE TABLE mentorship_posts (
    id          SERIAL PRIMARY KEY,
    user_id     INT         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type        VARCHAR(10) NOT NULL CHECK (type IN ('offer', 'request')),
    skill_id    INT         NOT NULL REFERENCES skills(id),
    mode        VARCHAR(10) NOT NULL CHECK (mode IN ('online', 'offline', 'both')),
    description TEXT,
    is_active   BOOLEAN DEFAULT TRUE,   -- FALSE pour archiver sans supprimer
    created_at  TIMESTAMP DEFAULT NOW(),
    updated_at  TIMESTAMP DEFAULT NOW()
);


-- ── TABLE: post_availabilities ───────────────────────────
-- Disponibilités horaires spécifiques à un post
CREATE TABLE post_availabilities (
    id          SERIAL PRIMARY KEY,
    post_id     INT         NOT NULL REFERENCES mentorship_posts(id) ON DELETE CASCADE,
    day_of_week VARCHAR(10) NOT NULL CHECK (day_of_week IN (
        'Monday','Tuesday','Wednesday','Thursday',
        'Friday','Saturday','Sunday'
    )),
    start_time  TIME NOT NULL,
    end_time    TIME NOT NULL,
    CONSTRAINT chk_post_avail_time CHECK (start_time < end_time)
);


-- ── TABLE: matches ───────────────────────────────────────
-- Correspondances mentor ↔ mentoré calculées par l'algorithme
CREATE TABLE matches (
    id              SERIAL PRIMARY KEY,
    mentor_id       INT           NOT NULL REFERENCES users(id),
    mentee_id       INT           NOT NULL REFERENCES users(id),
    offer_post_id   INT           REFERENCES mentorship_posts(id),
    request_post_id INT           REFERENCES mentorship_posts(id),
    skill_id        INT           REFERENCES skills(id),
    score           DECIMAL(5,2)  NOT NULL DEFAULT 0,
    status          VARCHAR(20)   NOT NULL DEFAULT 'pending'
                        CHECK (status IN ('pending', 'accepted', 'rejected')),
    created_at      TIMESTAMP DEFAULT NOW(),
    CONSTRAINT chk_mentor_mentee_diff CHECK (mentor_id <> mentee_id)
);


-- ── TABLE: conversations ─────────────────────────────────
-- Une conversation est créée pour chaque match accepté (1:1 avec matches)
CREATE TABLE conversations (
    id         SERIAL PRIMARY KEY,
    match_id   INT NOT NULL UNIQUE REFERENCES matches(id) ON DELETE CASCADE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);


-- ── TABLE: messages ──────────────────────────────────────
CREATE TABLE messages (
    id              SERIAL PRIMARY KEY,
    conversation_id INT     NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id       INT     NOT NULL REFERENCES users(id),
    content         TEXT    NOT NULL,
    is_read         BOOLEAN DEFAULT FALSE,
    created_at      TIMESTAMP DEFAULT NOW()
);


-- ── INDEX (pour meilleures performances des requêtes) ────
CREATE INDEX idx_user_skills_user     ON user_skills(user_id);
CREATE INDEX idx_user_skills_skill    ON user_skills(skill_id);
CREATE INDEX idx_avail_user           ON user_availabilities(user_id);
CREATE INDEX idx_posts_user           ON mentorship_posts(user_id);
CREATE INDEX idx_posts_skill          ON mentorship_posts(skill_id);
CREATE INDEX idx_posts_active         ON mentorship_posts(is_active);
CREATE INDEX idx_matches_mentor       ON matches(mentor_id);
CREATE INDEX idx_matches_mentee       ON matches(mentee_id);
CREATE INDEX idx_matches_status       ON matches(status);
CREATE INDEX idx_messages_conv        ON messages(conversation_id);
CREATE INDEX idx_messages_sender      ON messages(sender_id);
CREATE INDEX idx_messages_created     ON messages(created_at);


-- ── DONNÉES INITIALES: compétences IFRI ──────────────────
INSERT INTO skills (name) VALUES
    ('Algorithmique'),
    ('Structures de données'),
    ('Programmation Python'),
    ('Développement Web (HTML/CSS)'),
    ('JavaScript'),
    ('Vue.js'),
    ('Bases de données SQL'),
    ('Algèbre relationnelle'),
    ('Modélisation de bases de données'),
    ('Réseaux informatiques'),
    ('Systèmes d''exploitation Linux'),
    ('Mathématiques discrètes'),
    ('Intelligence Artificielle'),
    ('Probabilités et statistiques'),
    ('FastAPI / Flask / Django')
;
```

5 Architecture de l'application

L'application suit une architecture **client-serveur** classique : le frontend Vue.js (le client) communique avec le backend FastAPI (le serveur) via une **API REST en JSON**. Le backend accède à la base de données Supabase/PostgreSQL pour lire et écrire les données.

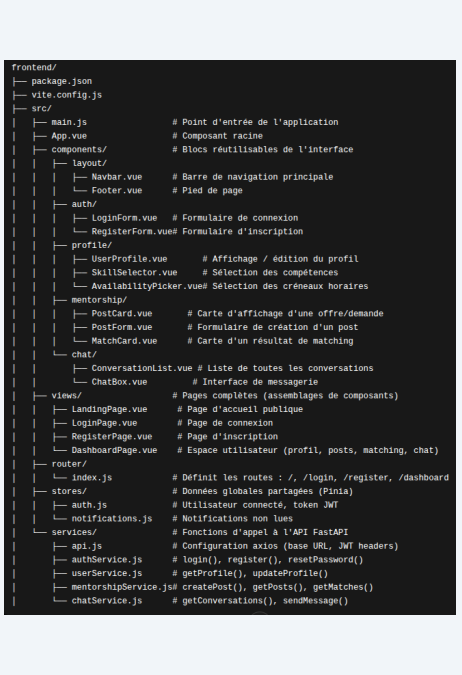
```
VUE D'ENSEMBLE DU FLUX

🖥 NAVIGATEUR (Vue.js)
    Construit l'interface, écoute les actions de l'utilisateur
  ↕ HTTP REST / JSON (axios)
⚡ SERVEUR API (FastAPI)
    Gère la logique métier, vérifie le JWT, appelle la BDD
  ↕ SQL (SQLAlchemy / psycopg2)
🗄 BASE DE DONNÉES (Supabase/PostgreSQL)
    Stocke et retourne toutes les données persistantes
  ↕ Realtime (WebSocket Supabase)
💬 MESSAGERIE EN TEMPS RÉEL
    Vue.js reçoit les nouveaux messages instantanément via Supabase Realtime
```

Structure du projet Frontend (Vue.js)

La structure suit les conventions officielles de Vue 3 avec Vite. Chaque dossier a un rôle précis.

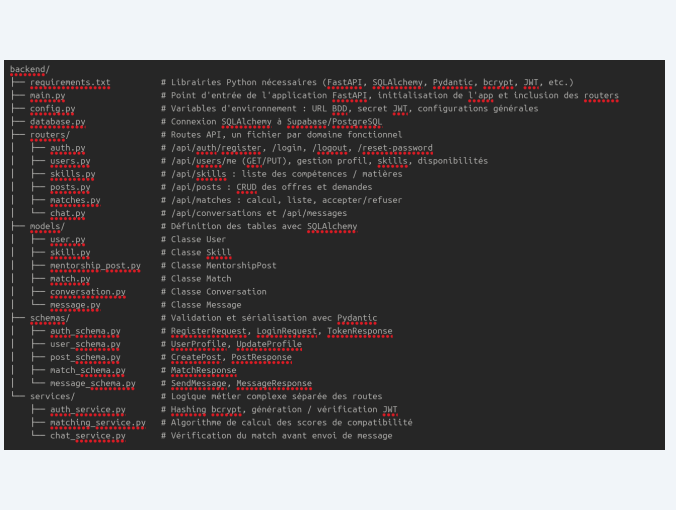
```
frontend/ ├── src/ │ ├── components/ — Blocs réutilisables de l'interface │ │ ├── layout/ │ │ │ ├── Navbar.vue — Barre de navigation principale │ │ │ └── Footer.vue — Pied de page │ │ ├── auth/ │ │ │ ├── LoginForm.vue — Formulaire de connexion │ │ │ └── RegisterForm.vue — Formulaire d'inscription │ │ ├── profile/ │ │ │ ├── UserProfile.vue — Affichage / édition du profil │ │ │ ├── SkillSelector.vue — Sélection des compétences │ │ │ └── AvailabilityPicker.vue — Sélection des créneaux horaires │ │ ├── mentorship/ │ │ │ ├── PostCard.vue — Carte d'affichage d'une offre/demande │ │ │ ├── PostForm.vue — Formulaire de création d'un post │ │ │ └── MatchCard.vue — Carte d'un résultat de matching │ │ └── chat/ │ │ ├── ConversationList.vue — Liste de toutes les conversations │ │ └── ChatBox.vue — Interface de messagerie avec messages │ ├── views/ — Pages complètes (assemblages de composants) │ │ ├── LandingPage.vue — Page d'accueil publique │ │ ├── LoginPage.vue — Page de connexion │ │ ├── RegisterPage.vue — Page d'inscription │ │ └── DashboardPage.vue — Espace utilisateur (profil, posts, matching, chat) │ ├── router/ │ │ └── index.js — Définit les routes : /, /login, /register, /dashboard │ ├── stores/ — Données globales partagées (Pinia) │ │ ├── auth.js — Utilisateur connecté, token JWT │ │ └── notifications.js — Notifications non lues │ ├── services/ — Fonctions d'appel à l'API FastAPI │ │ ├── api.js — Configuration axios (base URL, JWT headers) │ │ ├── authService.js — login(), register(), resetPassword() │ │ ├── userService.js — getProfile(), updateProfile() │ │ ├── mentorshipService.js — createPost(), getPosts(), getMatches() │ │ └── chatService.js — getConversations(), sendMessage() │ ├── App.vue — Composant racine │ └── main.js — Point d'entrée de l'application ├── package.json └── vite.config.js
```



```
frontend/
├── package.json
├── vite.config.js
├── src/
│   ├── main.js                # Point d'entrée de l'application
│   ├── App.vue                # Composant racine
│   ├── components/            # Blocs réutilisables de l'interface
│   │   ├── layout/
│   │   │   ├── Navbar.vue     # Barre de navigation principale
│   │   │   └── Footer.vue     # Pied de page
│   │   ├── auth/
│   │   │   ├── LoginForm.vue  # Formulaire de connexion
│   │   │   └── RegisterForm.vue# Formulaire d'inscription
│   │   ├── profile/
│   │   │   ├── UserProfile.vue      # Affichage / édition du profil
│   │   │   ├── SkillSelector.vue    # Sélection des compétences
│   │   │   └── AvailabilityPicker.vue# Sélection des créneaux horaires
│   │   ├── mentorship/
│   │   │   ├── PostCard.vue     # Carte d'affichage d'une offre/demande
│   │   │   ├── PostForm.vue     # Formulaire de création d'un post
│   │   │   └── MatchCard.vue    # Carte d'un résultat de matching
│   │   └── chat/
│   │       ├── ConversationList.vue # Liste de toutes les conversations
│   │       └── ChatBox.vue          # Interface de messagerie
│   ├── views/                # Pages complètes (assemblages de composants)
│   │   ├── LandingPage.vue   # Page d'accueil publique
│   │   ├── LoginPage.vue     # Page de connexion
│   │   ├── RegisterPage.vue  # Page d'inscription
│   │   └── DashboardPage.vue # Espace utilisateur (profil, posts, matching, chat)
│   ├── router/
│   │   └── index.js          # Définit les routes : /, /login, /register, /dashboard
│   ├── stores/               # Données globales partagées (Pinia)
│   │   ├── auth.js           # Utilisateur connecté, token JWT
│   │   └── notifications.js  # Notifications non lues
│   └── services/             # Fonctions d'appel à l'API FastAPI
│       ├── api.js            # Configuration axios (base URL, JWT headers)
│       ├── authService.js    # login(), register(), resetPassword()
│       ├── userService.js    # getProfile(), updateProfile()
│       ├── mentorshipService.js# createPost(), getPosts(), getMatches()
│       └── chatService.js    # getConversations(), sendMessage()
```

Structure du projet Backend (FastAPI)

```
backend/ ├── main.py — Point d'entrée FastAPI, initialisation de l'app ├── config.py — Variables d'environnement (URL BDD, secret JWT...) ├── database.py — Connexion SQLAlchemy à Supabase/PostgreSQL ├── routers/ — Un fichier par domaine fonctionnel │ ├── auth.py — /api/auth/register, /login, /logout, /reset-password │ ├── users.py — /api/users/me (GET/PUT), skills, availabilities │ ├── skills.py — /api/skills (liste de toutes les matières) │ ├── posts.py — /api/posts (CRUD offres et demandes) │ ├── matches.py — /api/matches (calcul, liste, accepter/refuser) │ └── chat.py — /api/conversations, /api/messages ├── models/ — Classes SQLAlchemy (représentent les tables) │ ├── user.py │ ├── skill.py │ ├── mentorship_post.py │ ├── match.py │ ├── conversation.py │ └── message.py ├── schemas/ — Classes Pydantic (validation des données entrantes/sortantes) │ ├── auth_schema.py — RegisterRequest, LoginRequest, TokenResponse │ ├── user_schema.py — UserProfile, UpdateProfile │ ├── post_schema.py — CreatePost, PostResponse │ ├── match_schema.py — MatchResponse │ └── message_schema.py — SendMessage, MessageResponse ├── services/ — Logique métier complexe (séparée des routes) │ ├── auth_service.py — Hashing bcrypt, génération/vérification JWT │ ├── matching_service.py — Algorithme de calcul des scores │ └── chat_service.py — Vérification du match avant envoi de message └── requirements.txt — Liste des librairies Python à installer
```



```
backend/
├── requirements.txt        # Librairies Python nécessaires (FastAPI, SQLAlchemy, Pydantic, bcrypt, JWT, etc.)
├── main.py                 # Point d'entrée de l'application FastAPI, initialisation de l'app et inclusion des routers
├── config.py               # Variables d'environnement : URL BDD, secret JWT, configurations générales
├── database.py             # Connexion SQLAlchemy à Supabase/PostgreSQL
├── routers/                # Routes API, un fichier par domaine fonctionnel
│   ├── auth.py             # /api/auth/register, /login, /logout, /reset-password
│   ├── users.py            # /api/users/me (GET/PUT), gestion profil, skills, disponibilités
│   ├── skills.py           # /api/skills : liste des compétences / matières
│   ├── posts.py            # /api/posts : CRUD des offres et demandes
│   ├── matches.py          # /api/matches : calcul, liste, accepter/refuser
│   └── chat.py             # /api/conversations et /api/messages
├── models/                 # Définition des tables avec SQLAlchemy
│   ├── user.py             # Classe User
│   ├── skill.py            # Classe Skill
│   ├── mentorship_post.py  # Classe MentorshipPost
│   ├── match.py            # Classe Match
│   ├── conversation.py     # Classe Conversation
│   └── message.py          # Classe Message
├── schemas/                # Validation et sérialisation avec Pydantic
│   ├── auth_schema.py      # RegisterRequest, LoginRequest, TokenResponse
│   ├── user_schema.py      # UserProfile, UpdateProfile
│   ├── post_schema.py      # CreatePost, PostResponse
│   ├── match_schema.py     # MatchResponse
│   └── message_schema.py   # SendMessage, MessageResponse
└── services/               # Logique métier complexe séparée des routes
    ├── auth_service.py     # Hashing bcrypt, génération / vérification JWT
    ├── matching_service.py # Algorithme de calcul des scores de compatibilité
    └── chat_service.py     # Vérification du match avant envoi de message
```

CONTENU DU REQUIREMENTS.TXT

```
fastapi
uvicorn[standard]
sqlalchemy
psycopg2-binary
python-jose[cryptography]
passlib[bcrypt]
python-dotenv
python-multipart
pydantic[email]
```

Endpoints de l'API REST

Voici la liste complète des routes HTTP que le backend FastAPI doit exposer. Le frontend Vue.js appelle ces routes via axios.

AUTHENTIFICATION

MÉTHODE	ROUTE	DESCRIPTION	AUTH REQUISE
POST	`/api/auth/register`	Inscription d'un nouvel utilisateur	Non
POST	`/api/auth/login`	Connexion, retourne un token JWT	Non
POST	`/api/auth/reset-password`	Réinitialisation du mot de passe	Non

PROFIL UTILISATEUR

MÉTHODE	ROUTE	DESCRIPTION	AUTH REQUISE
GET	`/api/users/me`	Récupérer le profil de l'utilisateur connecté	JWT
PUT	`/api/users/me`	Modifier les informations du profil	JWT
POST	`/api/users/me/skills`	Ajouter une compétence (strong/weak)	JWT
DELETE	`/api/users/me/skills/{skill_id}`	Supprimer une compétence du profil	JWT
POST	`/api/users/me/availabilities`	Ajouter un créneau de disponibilité	JWT
DELETE	`/api/users/me/availabilities/{id}`	Supprimer un créneau	JWT

COMPÉTENCES (RÉFÉRENCE)

MÉTHODE	ROUTE	DESCRIPTION	AUTH REQUISE
GET	`/api/skills`	Liste de toutes les compétences disponibles	Non

POSTS DE MENTORAT

MÉTHODE	ROUTE	DESCRIPTION	AUTH REQUISE
GET	`/api/posts`	Lister tous les posts actifs (filtrables par skill, type)	JWT
POST	`/api/posts`	Créer un nouveau post (offre ou demande)	JWT
GET	`/api/posts/{id}`	Détails d'un post spécifique	JWT
PUT	`/api/posts/{id}`	Modifier un post (seulement le sien)	JWT
DELETE	`/api/posts/{id}`	Supprimer ou archiver un post	JWT

MATCHING

MÉTHODE	ROUTE	DESCRIPTION	AUTH REQUISE
GET	`/api/matches`	Obtenir mes correspondances calculées (triées par score)	JWT
PUT	`/api/matches/{id}/accept`	Accepter un match → crée une conversation	JWT
PUT	`/api/matches/{id}/reject`	Refuser un match	JWT

MESSAGERIE

MÉTHODE	ROUTE	DESCRIPTION	AUTH REQUISE
GET	`/api/conversations`	Liste de mes conversations (avec dernier message)	JWT
GET	`/api/conversations/{id}/messages`	Historique des messages d'une conversation	JWT
POST	`/api/conversations/{id}/messages`	Envoyer un message (vérifie match accepté)	JWT
PUT	`/api/messages/{id}/read`	Marquer un message comme lu	JWT

Authentification avec JWT

JWT (JSON Web Token) est la façon standard de gérer les sessions dans les APIs modernes. Voici comment ça fonctionne dans IFRI_MentorLink :

1 **L'utilisateur envoie son email + mot de passe à** `POST /api/auth/login`

FastAPI reçoit la requête. Il cherche l'utilisateur par email en BDD, puis vérifie que le mot de passe fourni correspond au hash bcrypt stocké via la fonction `passlib.verify()` .

2 **FastAPI génère un token JWT signé**

Le token contient l'ID de l'utilisateur et une date d'expiration (ex: 24h). Il est **signé** avec une clé secrète définie dans `config.py` . Personne ne peut falsifier ce token sans connaître la clé secrète.

```python
# Exemple simplifié de génération JWT
from jose import jwt

def create_token(user_id: int) -> str:
    payload = {
        "sub": str(user_id),          # ID de l'utilisateur
        "exp": datetime.utcnow() + timedelta(hours=24)  # Expiration
    }
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

3 **Le frontend Vue.js stocke le token et l'envoie à chaque requête**

Vue.js stocke le token dans le store Pinia et l'envoie dans le header HTTP `Authorization: Bearer <token>` à chaque appel API protégé.

4 **FastAPI vérifie le token à chaque requête protégée**

Les routes qui nécessitent une connexion utilisent une dépendance FastAPI (`get_current_user`) qui lit et vérifie le token automatiquement. Si le token est invalide ou expiré, la requête est rejetée avec une erreur 401.

Algorithme de matching

L'algorithme de matching est le cœur de la plateforme. Il calcule un **score de compatibilité sur 100 points** pour chaque paire mentor-mentoré potentielle, selon 3 critères.

CRITÈRE 1
Compétences — 40 pts
Le mentor a la matière en 'strong', le mentoré l'a en 'weak' → +40 points

CRITÈRE 2
Disponibilités — 30 pts max
Créneaux horaires communs. +10 pts par créneau partagé, maximum 30 pts.

CRITÈRE 3
Filière / Niveau — 30 pts max
Même filière → +30 pts. Filières liées → +15 pts. Différentes → +0 pt.

```python
# matching_service.py — Logique de l'algorithme
# Score maximum possible : 40 + 30 + 30 = 100 points

def calculate_score(mentor, mentee, skill_id: int) -> float:
    score = 0.0

    # ── CRITÈRE 1 : Compatibilité des compétences (40 pts max) ──
    # Vérifier que le mentor maîtrise la compétence (strong)
    mentor_strong = {us.skill_id for us in mentor.skills if us.proficiency == 'strong'}
    # Vérifier que le mentoré en a besoin (weak)
    mentee_weak = {us.skill_id for us in mentee.skills if us.proficiency == 'weak'}

    if skill_id in mentor_strong and skill_id in mentee_weak:
        score += 40   # Correspondance parfaite

    # ── CRITÈRE 2 : Disponibilités communes (30 pts max) ──
    # Trouver les créneaux qui se chevauchent entre les deux utilisateurs
    common_slots = 0
    for m_avail in mentor.availabilities:
        for t_avail in mentee.availabilities:
            if (m_avail.day_of_week == t_avail.day_of_week and
                m_avail.start_time < t_avail.end_time and
                t_avail.start_time < m_avail.end_time):
                common_slots += 1  # Ce créneau est commun

    score += min(30, common_slots * 10)  # +10 pts par créneau, max 30

    # ── CRITÈRE 3 : Proximité de filière (30 pts max) ──
    RELATED_FIELDS = {
        'IA': ['IM', 'GL'],        # IA est proche de IM et GL
        'IM': ['IA', 'GL'],
        'GL': ['IA', 'IM', 'SI'],
        'SI': ['GL'],
        'SE&IoT': ['IA']
    }
    if mentor.field_of_study == mentee.field_of_study:
        score += 30   # Même filière
    elif mentee.field_of_study in RELATED_FIELDS.get(mentor.field_of_study, []):
        score += 15   # Filières liées

    return round(score, 2)


def compute_matches_for_user(db, current_user_id: int):
    """
    Calcule tous les matches possibles pour un utilisateur donné
    et les insère dans la table matches.
    """
    user = db.query(User).filter(User.id == current_user_id).first()
    results = []

    # Parcourir tous les autres utilisateurs actifs
    all_others = db.query(User).filter(User.id != current_user_id).all()

    for other in all_others:
        # Pour chaque compétence en commun (offer ↔ request)
        user_weak_skills = {us.skill_id for us in user.skills if us.proficiency == 'weak'}
        other_strong_skills = {us.skill_id for us in other.skills if us.proficiency == 'strong'}
        common = user_weak_skills & other_strong_skills  # Intersection

        for skill_id in common:
            score = calculate_score(
                mentor=other, mentee=user, skill_id=skill_id
            )
            if score >= 40:  # Seuil minimum pour proposer un match
                results.append({
                    'mentor_id': other.id,
                    'mentee_id': user.id,
                    'skill_id': skill_id,
                    'score': score,
                    'status': 'pending'
                })

    # Trier par score décroissant
    results.sort(key=lambda x: x['score'], reverse=True)
    return results
```

Messagerie en temps réel avec Supabase Realtime

Supabase offre une fonctionnalité de **Realtime** : il peut notifier le frontend **automatiquement** dès qu'une ligne est insérée dans une table, via WebSocket. C'est beaucoup plus simple que d'implémenter des WebSockets manuellement dans FastAPI.

Comment ça marche côté Vue.js : au lieu de faire des requêtes GET toutes les secondes pour chercher de nouveaux messages (polling), Vue.js *écoute* Supabase. Dès qu'un message est inséré en BDD, Supabase le pousse automatiquement vers Vue.js.

```javascript
// chatService.js — Écoute des nouveaux messages en temps réel
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)

// Cette fonction s'abonne aux nouveaux messages d'une conversation
export function subscribeToMessages(conversationId, onNewMessage) {
  const subscription = supabase
    .channel(`messages:conv:${conversationId}`)
    .on(
      'postgres_changes',
      {
        event: 'INSERT',        // Écouter uniquement les insertions
        schema: 'public',
        table: 'messages',
        filter: `conversation_id=eq.${conversationId}`
      },
      (payload) => {
        // Cette fonction est appelée automatiquement à chaque nouveau message
        onNewMessage(payload.new)
      }
    )
    .subscribe()

  return subscription   // Garder la référence pour se désabonner plus tard
}

// Usage dans ChatBox.vue :
// const sub = subscribeToMessages(convId, (msg) => messages.push(msg))
// onUnmounted(() => sub.unsubscribe())
```

6 Répartition des tâches et workflow Git

L'équipe est composée de **6 membres** : 2 développeurs frontend et 4 développeurs backend. Cette répartition est justifiée par la complexité du backend (authentification, algorithme de matching, messagerie). **Chaque membre travaille sur une branche Git dédiée** et ne travaille que sur sa partie.

Workflow Git — 3 niveaux de branches

```
── Structure des branches ── main ── Code stable, prêt à être évalué. Merge UNIQUEMENT en fin de projet └── develop ── Branche d'intégration, tout le monde merge ici après ses features ├── feature/landing-auth ── Personne A (Frontend 1) ├── feature/dashboard ── Personne B (Frontend 2) ├── feature/auth-backend ── Personne C (Backend 1) ├── feature/users-skills ── Personne D (Backend 2) ├── feature/posts-matching ── Personne E (Backend 3) └── feature/chat ── Personne F (Backend 4)
```

RÈGLES IMPÉRATIVES DU WORKFLOW

Interdictions absolues :

- Ne jamais committer directement sur `main` ou `develop`
- Ne jamais modifier les fichiers d'une autre personne sans la prévenir
- Ne jamais pousser du code qui **casse** le build sur `develop`

ROUTINE QUOTIDIENNE POUR CHAQUE DEV

```
# Début de journée : récupérer les derniers changements
git checkout develop
git pull origin develop
git checkout feature/ma-branche
git merge develop   # Intégrer les nouveautés

# Pendant le travail : committer souvent
git add .
git commit -m "feat: description courte de ce que j'ai fait"
git push origin feature/ma-branche

# Quand une feature est terminée
# → Ouvrir une Pull Request vers develop sur GitHub
```

CONVENTIONS DE COMMIT

- ✓ `feat:` nouvelle fonctionnalité
- ✓ `fix:` correction de bug
- ✓ `style:` changements CSS / design
- ✓ `db:` modification de la BDD
- ✓ `docs:` documentation / rapport
- ✓ `chore:` config, setup, etc.

Exemple : `feat: add JWT authentication endpoint`

Fiches détaillées par personne

A **Personne A — Frontend 1** `feature/landing-auth`
Landing Page · Authentification · Intégration auth API

CE QUE TU CONSTRUIS

La première impression de IFRI_MentorLink. La landing page donne envie de s'inscrire, et les formulaires d'inscription/connexion permettent d'accéder à la plateforme de façon sécurisée.

TÂCHES DÉTAILLÉES

- ✓ **LandingPage.vue** : Navbar avec liens vers les sections, Hero section avec slogan et illustration, section "À propos" expliquant le projet, et boutons CTA "S'inscrire" / "Se connecter"
- ✓ **RegisterPage.vue + RegisterForm.vue** : Formulaire complet avec tous les champs requis (nom, prénom, email, téléphone, mot de passe, filière, niveau), validation côté client (champs obligatoires, format email, force du mot de passe)
- ✓ **LoginPage.vue + LoginForm.vue** : Formulaire de connexion (email ou téléphone + mot de passe), lien "Mot de passe oublié", gestion des erreurs affichées à l'utilisateur
- ✓ **stores/auth.js (Pinia)** : Stocker le token JWT reçu après connexion, stocker les infos de l'utilisateur connecté, fonction de déconnexion (efface le token)
- ✓ **services/authService.js** : Fonctions `login()` et `register()` qui font les appels axios vers l'API FastAPI
- ✓ **router/index.js** : Définir toutes les routes (`/` , `/login` , `/register` , `/dashboard`), mettre une protection sur `/dashboard` (rediriger vers /login si pas de token)

FICHIERS CLÉS À CRÉER

`src/views/LandingPage.vue` `src/views/LoginPage.vue` `src/views/RegisterPage.vue` `src/components/auth/LoginForm.vue` `src/components/auth/RegisterForm.vue` `src/stores/auth.js` `src/services/authService.js` `src/router/index.js`

DÉPENDANCES

Tu as besoin que **Personne C (Back 1)** ait terminé les endpoints `/api/auth/register` et `/api/auth/login` pour tester l'intégration. Commence avec des données mockées (fausses) le temps que le back soit prêt.

CRITÈRE DE "C'EST TERMINÉ"

Un étudiant peut atterrir sur la landing page, cliquer sur S'inscrire, remplir le formulaire, être redirigé vers /dashboard. Et depuis /dashboard, cliquer sur Déconnexion revient vers /login.

B **Personne B — Frontend 2** `feature/dashboard`
Dashboard · Profil · Posts · Matching · Chat UI

CE QUE TU CONSTRUIS

L'espace utilisateur complet. Après la connexion, tout se passe ici. C'est la partie la plus riche de l'interface et elle doit être intuitive, claire et bien organisée par onglets.

TÂCHES DÉTAILLÉES

- ✓ **DashboardPage.vue** : Page principale avec navigation par onglets (Profil, Mes Posts, Mes Matches, Messagerie). Récupère et affiche le nom de l'utilisateur connecté dans la navbar.
- ✓ **UserProfile.vue** : Affichage et édition des infos du profil (nom, bio, filière, niveau). Appel à `GET /api/users/me` puis `PUT /api/users/me` pour les modifications.
- ✓ **SkillSelector.vue** : Interface pour ajouter/supprimer des compétences (points forts et faibles) depuis la liste des skills. Appels à `POST/DELETE /api/users/me/skills` .
- ✓ **AvailabilityPicker.vue** : Interface pour choisir ses créneaux (jour + heure début + heure fin). Appels à `POST/DELETE /api/users/me/availabilities` .
- ✓ **PostCard.vue + PostForm.vue** : Afficher les offres/demandes existantes et formulaire pour en créer de nouvelles. Appels à `GET/POST /api/posts` .
- ✓ **MatchCard.vue** : Afficher une correspondance proposée avec nom, photo (si disponible), compétences communes, horaires partagés et score. Boutons "Accepter" et "Refuser".
- ✓ **ConversationList.vue** : Liste des conversations actives avec dernier message et indicateur de messages non lus.
- ✓ **ChatBox.vue** : Interface de messagerie avec historique des messages et champ d'envoi. Intégration du **Supabase Realtime** pour les nouveaux messages en temps réel.

FICHIERS CLÉS À CRÉER

`src/views/DashboardPage.vue` `src/components/profile/UserProfile.vue` `src/components/profile/SkillSelector.vue` `src/components/profile/AvailabilityPicker.vue` `src/components/mentorship/PostCard.vue` `src/components/mentorship/PostForm.vue` `src/components/mentorship/MatchCard.vue` `src/components/chat/ConversationList.vue` `src/components/chat/ChatBox.vue`

DÉPENDANCES

Tu dépends de l'ensemble des backends. Travaille d'abord sur la structure visuelle (HTML/CSS) et les composants statiques avec des données fausses. Intègre les vrais appels API au fur et à mesure que les backends sont terminés.

C **Personne C — Backend 1** `feature/auth-backend`
Authentification · JWT · Sécurité

CE QUE TU CONSTRUIS

Le système de sécurité de toute l'application. Sans toi, personne ne peut se connecter. C'est le module le plus critique : s'il a un bug, tout le reste est bloqué. Priorité absolue.

TÂCHES DÉTAILLÉES

- ✓ **database.py** : Configurer la connexion SQLAlchemy à Supabase. Créer la session de base de données (fonction `get_db()` utilisée comme dépendance dans toutes les routes).
- ✓ **models/user.py** : Classe SQLAlchemy représentant la table `users` .
- ✓ **services/auth_service.py** : Fonctions de hachage du mot de passe avec bcrypt (`hash_password()` et `verify_password()`), fonctions de génération et vérification du JWT (`create_token()` et `verify_`
- ✓ **routers/auth.py** : Route `POST /api/auth/register` (vérifier unicité email/téléphone, hasher le MDP, créer l'utilisateur), route `POST /api/auth/login` (vérifier les identifiants, retourner le token), route `POST /api/auth/reset-password`
- ✓ **Dépendance `get_current_user()`** : Fonction FastAPI qui lit le token JWT dans le header, le vérifie et retourne l'utilisateur courant. Utilisée par TOUTES les autres routes protégées.
- ✓ **main.py** : Initialiser FastAPI, configurer CORS (pour que Vue.js puisse appeler l'API), inclure tous les routers.

FICHIERS CLÉS À CRÉER

`backend/main.py` `backend/database.py` `backend/config.py` `backend/services/auth_service.py` `backend/models/user.py` `backend/schemas/auth_schema.py` `backend/routers/auth.py`

C'EST FINI QUAND

Sur Swagger UI (accessible à `http://localhost:8000/docs`), tu peux appeler `/register` , puis `/login` , obtenir un token, et utiliser ce token pour appeler une route protégée de test. Le mot de passe est haché en BDD (jamais en clair).

D **Personne D — Backend 2** `feature/users-skills`
Profils utilisateurs · Compétences · Disponibilités

CE QUE TU CONSTRUIS

La gestion complète du profil utilisateur. Tout ce qui concerne les données personnelles, les compétences (points forts et faibles) et les disponibilités horaires.

TÂCHES DÉTAILLÉES

- ✓ **routers/users.py** : `GET /api/users/me` (retourner le profil complet avec skills et disponibilités), `PUT /api/users/me` (modifier les infos de base : nom, bio, filière, niveau, photo).
- ✓ **routers/skills.py** : `GET /api/skills` (retourner la liste complète des compétences depuis la table `skills`), `POST /api/users/me/skills` (ajouter une compétence avec son niveau strong/weak), `DELETE /api/users/me/skills`
- ✓ **Endpoints disponibilités** : `POST /api/users/me/availabilities` (ajouter un créneau : jour, heure début, heure fin), `DELETE /api/users/me/availabilities/{id}` (supprimer un créneau).
- ✓ **models/skill.py + user_skill.py + user_availability.py** : Les modèles SQLAlchemy correspondant aux tables.
- ✓ **schemas/user_schema.py** : Les classes Pydantic pour valider et formater les données (ex: `UserProfileResponse` , `UpdateProfileRequest` , `AddSkillRequest`).

FICHIERS CLÉS À CRÉER

`backend/routers/users.py` `backend/routers/skills.py` `backend/models/skill.py` `backend/schemas/user_schema.py`

DÉPENDANCES

Tu as besoin que **Personne C** ait fini la dépendance `get_current_user()` pour protéger tes routes. Commence à coder les modèles et schémas en attendant.

E **Personne E — Backend 3** `feature/posts-matching`
Posts de mentorat · Algorithme de matching

CE QUE TU CONSTRUIS

La fonctionnalité principale de la plateforme : les offres/demandes de mentorat et l'algorithme qui propose automatiquement les bonnes correspondances.

TÂCHES DÉTAILLÉES

- ✓ **routers/posts.py** : `GET /api/posts` avec filtres optionnels (par skill_id, par type offer/request, par mode), `POST /api/posts` pour créer un nouveau post, `PUT /api/posts/{id}` pour modifier (vérifier que c'est bien le post de l'utilisateur connecté), `DELETE /api/posts/{id}` po
- ✓ **post_availabilities dans routers/posts.py** : Lors de la création d'un post, accepter également une liste de créneaux horaires et les insérer dans `post_availabilities` .
- ✓ **services/matching_service.py** : Implémenter la fonction `calculate_score()` et `compute_matches_for_user()` décrites dans la section 5. Le code est fourni dans ce document.
- ✓ **routers/matches.py** : `GET /api/matches` qui appelle le service de matching et retourne les résultats triés, `PUT /api/matches/{id}/accept` (change le status en 'accepted' et crée automatiquement une conversation), `PUT /api/matches/{id}/reje`
- ✓ **models/mentorship_post.py + match.py** : Les modèles SQLAlchemy correspondants.

FICHIERS CLÉS À CRÉER

`backend/routers/posts.py` `backend/routers/matches.py` `backend/services/matching_service.py` `backend/models/mentorship_post.py` `backend/models/match.py` `backend/schemas/post_schema.py` `backend/schemas/match_schema.py`

⚠ POINT D'ATTENTION

L'algorithme de matching est la partie la plus complexe du backend. Le code de base est fourni dans la section 5 de ce document. Lis-le bien et adapte-le à ta structure de modèles SQLAlchemy. Pose des questions si quelque chose n'est pas clair.

F **Personne F — Backend 4** `feature/chat`
Conversations · Messages · Notifications temps réel

CE QUE TU CONSTRUIS

Le module de messagerie. Une fois que deux utilisateurs ont un match accepté, tu fais en sorte qu'ils puissent communiquer et que les messages arrivent en temps réel.

TÂCHES DÉTAILLÉES

- ✓ **routers/chat.py** : `GET /api/conversations` (liste des conversations de l'utilisateur connecté, avec le dernier message et le nombre de messages non lus), `GET /api/conversations/{id}/messages` (historique complet des messages), `POST /api/conversat`
- ✓ **Sécurité de la messagerie** : Avant d'autoriser l'envoi d'un message, vérifier que : (1) la conversation existe, (2) le match associé a le status 'accepted', (3) l'utilisateur connecté est bien le mentor ou le mentoré de ce match. Si une de ces conditions échoue → retourner une erreur 403.
- ✓ **Configuration Supabase Realtime** : Dans Supabase, activer la réplication sur la table `messages` (nécessaire pour que le Realtime fonctionne côté Vue.js). Instructions à fournir dans le README.
- ✓ **models/conversation.py + message.py** : Les modèles SQLAlchemy.

FICHIERS CLÉS À CRÉER

`backend/routers/chat.py` `backend/services/chat_service.py` `backend/models/conversation.py` `backend/models/message.py` `backend/schemas/message_schema.py`

DÉPENDANCES

Tu dépends de **Personne C** pour la dépendance `get_current_user()` et de **Personne E** pour la table `matches` et la logique d'acceptation qui crée les conversations. Commence par les modèles et schémas, puis code les routes quand E a fini l'acceptation de match.

Planning sur 4 jours

Ce planning est indicatif mais les deadlines sont réelles. **Si une tâche prend du retard, le signaler immédiatement** pour que l'équipe puisse réorganiser. Il vaut mieux demander de l'aide tôt que de bloquer tout le groupe.

JOUR	FRONTEND (A + B)	BACKEND (C + D + E + F)
Vendredi Aujourd'hui	Tout le monde : créer le repo GitHub, branches, installer les outils Tout le monde : exécuter le script SQL dans Supabase A : Structure de base Vue (vite create), Navbar, structure routing B : DashboardPage structure (onglets vides, composants mockés)	C : Setup FastAPI, database.py, main.py avec CORS, auth_service.py (bcrypt + JWT) D : models/ (user, skill, user_skill, user_availability) E : models/ (mentorship_post, match, post_availability) F : models/ (conversation, message)
Samedi Développement core	A : Landing page complète (Navbar + Hero + About + Footer) A : RegisterForm.vue + LoginForm.vue avec validation B : UserProfile.vue + SkillSelector.vue + AvailabilityPicker.vue	C : Terminer routers/auth.py (register + login + get_current_user) D : routers/users.py (GET/PUT /me) + routers/skills.py E : routers/posts.py (CRUD complet) F : routers/chat.py (GET conversations + messages)
Dimanche Intégration	A : Intégration auth (register + login qui appellent vraiment l'API) B : PostCard.vue + PostForm.vue intégrés à l'API B : MatchCard.vue (affichage + accepter/refuser) B : ChatBox.vue avec Supabase Realtime Merge de toutes les branches vers develop	E : matching_service.py terminé + routers/matches.py F : POST message + PUT read + sécurité messagerie Tous : Tests des endpoints sur Swagger Tous : Correction des bugs d'intégration
Lundi Finition + Rapport	Corrections de bugs UI signalés pendant les tests Amélioration de l'ergonomie et du design Rédaction du rapport HTML	Corrections de bugs backend remontés Export du fichier .sql depuis Supabase Tests end-to-end du parcours utilisateur complet Merge de develop → main

7 Livrables attendus

Le dépôt GitHub nommé `PIL1_2526_[XX]` doit contenir les éléments suivants au moment du rendu (mercredi 10 juin 2026 à 23h59).

RAPPORT HTML

- ✓ Nom du groupe et liste des membres
- ✓ Rôle et contribution de chaque membre
- ✓ Description de la conception (BDD, architecture)
- ✓ Instructions de déploiement
- ✓ Manuel d'utilisation

CODE SOURCE

- ✓ Dossier `frontend/` (Vue.js)
- ✓ Dossier `backend/` (FastAPI)
- ✓ Fichier `.env.example` (variables d'env sans secrets)
- ✓ `README.md` avec instructions de setup

FICHIER SQL

- ✓ Fichier `database.sql` à la racine du repo
- ✓ Script de création de toutes les tables
- ✓ Données initiales (skills)
- ✓ Correspond au schéma final utilisé en production

⚠ **Points clés pour la note :**

- **Tous les membres doivent avoir commité** sur le dépôt (vérifié dans l'historique Git). Un membre sans commit = problème.
- Les comptes GitHub `ratheilesse` , `primearwyn` et `MaryseGAHOU` doivent être collaborateurs du dépôt.
- Le dépôt doit s'appeler exactement `PIL1_2526_[NUMERO]` (ex: `PIL1_2526_03`).
- Les mots de passe **doivent être hachés** (aucun mot de passe en clair en BDD).

🏆 **Pour viser 16-20/20 :** Tout fonctionne, le design est soigné, le code est bien structuré, le rapport est clair et complet, tous les membres ont des commits réguliers, et l'algorithme de matching produit des résultats pertinents et bien présentés.

IFRI_MentorLink — Guide Technique — Projet Intégrateur PIL1 2025-2026 · IFRI / Université d'Abomey-Calavi